BÀI 9. LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OOP)

9.1. Class & Object

9.1.1. Khái niệm lập trình hướng đối tượng (OOP):

Lập trình hướng đối tượng (OOP) là một mô hình lập trình dựa trên khái niệm "đối tượng" (objects), có thể chứa dữ liệu (dưới dạng thuộc tính - attributes) và mã (dưới dạng phương thức - methods). Mục tiêu của OOP là tạo ra các chương trình dễ hiểu hơn, linh hoạt hơn và dễ bảo trì hơn.

Ưu điểm của OOP:

- **Tính mô-đun hóa (Modularity):** Chia nhỏ chương trình thành các đối tượng độc lập, dễ quản lý và phát triển riêng biệt.
- **Tính tái sử dụng (Reusability):** Các lớp (classes) có thể được định nghĩa một lần và sử dụng lại nhiều lần trong các phần khác nhau của chương trình hoặc trong các dự án khác.
- **Tính dễ bảo trì (Maintainability):** Khi có lỗi hoặc cần thay đổi, bạn chỉ cần sửa đổi trong lớp liên quan mà không ảnh hưởng đến các phần khác của chương trình.
- **Tính mở rộng (Extensibility):** Dễ dàng thêm các tính năng mới hoặc mở rộng chức năng hiện có mà không cần thay đổi mã nguồn ban đầu quá nhiều (thông qua kế thừa và đa hình).
- **Quản lý phức tạp (Complexity Management):** Giúp tổ chức và quản lý các hệ thống lớn, phức tạp bằng cách tạo ra các cấu trúc rõ ràng và dễ hiểu.

9.1.2. Class: bản thiết kế, khuôn mẫu cho đối tượng

Một **class** (lớp) giống như một **bản thiết kế, một khuôn mẫu, hoặc một kế hoạch chi tiết** để tạo ra các đối tượng. Nó định nghĩa các đặc điểm (thuộc tính) và hành vi (phương thức) mà các đối tượng thuộc lớp đó sẽ có. class không chiếm bộ nhớ thực sự để lưu trữ dữ liệu cụ thể, nó chỉ là một định nghĩa.

Cú pháp khai báo class: Bắt đầu bằng từ khóa class, theo sau là tên lớp (thường viết hoa chữ cái đầu tiên theo quy ước PascalCase).

```
class MyClass:
    # Các thuộc tính và phương thức sẽ được định nghĩa ở đây
    pass # 'pass' là một câu lệnh trống, dùng khi bạn chưa có gì để định
nghĩa
```

9.1.3. Object (đối tượng): thể hiện cụ thể của class

Một **Object** (đối tượng) là một **thể hiện cụ thể (instance)** của một class. Khi bạn tạo một đối tượng từ một class, nó sẽ chiếm bộ nhớ và mang các thuộc tính và phương thức được định nghĩa trong class đó. Bạn có thể tạo nhiều đối tượng từ cùng một class, mỗi đối tượng có thể có các giá trị thuộc tính riêng biệt.

Cách tạo Object: Gọi tên class như một hàm.

```
# Tạo đối tượng từ class MyClass
obj1 = MyClass()
obj2 = MyClass()
```

- obj1 và obj2 là hai đối tượng riêng biệt, được tạo ra từ cùng một bản thiết kế MyClass.

9.1.4. Thuộc tính (Attributes) và Phương thức (Methods)

1./ Thuộc tính (Attributes):

- Là các **biến** bên trong một class, dùng để lưu trữ dữ liệu (đặc điểm) của đối tượng. Chúng thường được định nghĩa trong hàm khởi tạo (__init__) của class (sẽ học sau).
- Thuộc tính dữ liệu (Data Attributes): Biểu diễn trạng thái của đối tượng.
- Cách truy cập thuộc tính: Sử dụng toán tử dấu chấm (.). object_name.attribute_name

2./ Phương thức (Methods):

- Là các **hàm** bên trong một class, dùng để định nghĩa hành vi (các hành động) mà đối tượng có thể thực hiện.
- Phương thức luôn có ít nhất một tham số đầu tiên, theo quy ước là self.
- **Cách gọi phương thức:** Sử dụng toán tử dấu chấm (.). object_name.method_name(arguments)

Ví dụ:

```
class Dog:
    # Thuộc tính lớp (Class attribute) - chung cho tất cả đối tượng Dog
    species = "Canis familiaris"

    # Phương thức khởi tạo (Constructor) - sẽ học kỹ hơn ở tiết sau
    # Dùng để khởi tạo thuộc tính của từng đối tượng
    def __init__(self, name, age):
        # Thuộc tính đối tượng (Instance attributes)
        self.name = name # Tên của con chó cụ thể này
```

```
        self.age = age   # Tuổi của con chó cụ thể này

    # Phương thức (Method)
    def bark(self):
        print(f"{self.name} says Woof!")

    def get_age_in_dog_years(self):
        return self.age * 7

# Tạo đối tượng (instantiate objects)
my_dog = Dog("Buddy", 3)
your_dog = Dog("Lucy", 5)

# Truy cập thuộc tính
print(f"Tên của chó tôi: {my_dog.name}") # Output: Tên của chó tôi: Buddy
print(f"Tuổi của chó bạn: {your_dog.age}")# Output: Tuổi của chó bạn: 5
print(f"Loài của chó tôi: {my_dog.species}") # Output: Loài của chó tôi:
Canis familiaris

# Gọi phương thức
my_dog.bark()                   # Output: Buddy says Woof!
your_dog.bark()                 # Output: Lucy says Woof!
print(f"Tuổi chó của Buddy theo tuổi chó:
{my_dog.get_age_in_dog_years()}") # Output: Tuổi chó của Buddy theo tuổi
chó: 21
```

Từ khóa self

- self là tham số đầu tiên của mỗi phương thức trong class (bao gồm cả __init__).
- self là một quy ước (không bắt buộc về tên, nhưng rất mạnh về ý nghĩa) và đại diện cho **chính đối tượng hiện tại** mà phương thức đang được gọi trên đó.
- Khi bạn gọi một phương thức, Python tự động truyền đối tượng hiện tại vào tham số self. Bạn không cần phải truyền nó một cách rõ ràng.
- Bạn sử dụng self để truy cập các thuộc tính và gọi các phương thức khác của chính đối tượng đó từ bên trong class.

Ví dụ: Trong def bark(self): print(f"{self.name} says Woof!"), self.name nghĩa là "lấy thuộc tính name của đối tượng đang gọi phương thức bark này".

9.2. Constructor & Kế thừa

9.2.1. Hàm khởi tạo (__init__): dùng để khởi tạo thuộc tính của đối tượng

1./ **Hàm khởi tạo (Constructor)** là một phương thức đặc biệt trong Python có tên là __init__ (có **hai dấu gạch dưới** ở đầu và cuối).

- Nó được gọi **tự động** ngay khi một đối tượng mới được tạo từ class.
- Mục đích chính của __init__ là để **khởi tạo trạng thái ban đầu** (tức là gán giá trị cho các thuộc tính) của đối tượng.
- Giống như các phương thức khác, __init__ nhận self làm tham số đầu tiên, theo sau là các tham số khác cần thiết để khởi tạo đối tượng.

Cú pháp:

```
class ClassName:
    def __init__(self, param1, param2, ...):
        self.attribute1 = param1
        self.attribute2 = param2
        # ...
```

Ví dụ:

```
class Person:
    def __init__(self, name, age):
        self.name = name  # Khởi tạo thuộc tính 'name' cho đối tượng
        self.age = age    # Khởi tạo thuộc tính 'age' cho đối tượng
        print(f"Đối tượng Person '{self.name}' đã được tạo.")

    def introduce(self):
        print(f"Xin chào, tôi là {self.name} và tôi {self.age} tuổi.")

# Khi tạo đối tượng, __init__ được gọi và bạn phải truyền các đối số cần
thiết
p1 = Person("Lan", 28)
p2 = Person("Minh", 35)

p1.introduce()
p2.introduce()
```

9.2.2. Kế thừa (Inheritance): khái niệm, lớp cha (Parent Class), lớp con (Child Class)

Kế thừa (Inheritance) là một trong những trụ cột của OOP, cho phép một class mới (gọi là **lớp con - Child Class** hoặc Derived Class) kế thừa các thuộc tính và phương thức từ một class hiện có (gọi là **lớp cha - Parent Class** hoặc Base Class/Superclass).

Lợi ích của kế thừa:

- **Tái sử dụng mã:** Tránh việc phải viết lại cùng một mã. Lớp con có thể sử dụng lại các thuộc tính và phương thức của lớp cha.
- Tạo ra mối quan hệ "là một" (is-a relationship):
  - Ví dụ: "Chó là một động vật", "Ô tô là một phương tiện". Điều này giúp cấu trúc chương trình rõ ràng và logic.

Cú pháp kế thừa: Đặt tên lớp cha trong dấu ngoặc đơn sau tên lớp con.

```
class ParentClass:
    # Thuộc tính và phương thức của lớp cha
    pass
```

```
class ChildClass(ParentClass):
    # Thuộc tính và phương thức riêng của lớp con
    # Có thể sử dụng lại hoặc ghi đè các phần từ lớp cha
    pass
```

Ví dụ:

```
class Animal: # Lớp cha
    def __init__(self, name):
        self.name = name
        print(f"Một con vật tên '{self.name}' đã được tạo.")

    def eat(self):
        print(f"{self.name} đang ăn.")

    def sleep(self):
        print(f"{self.name} đang ngủ.")

class Dog(Animal): # Lớp con kế thừa từ Animal
    def __init__(self, name, breed):
        super().__init__(name) # Gọi constructor của lớp cha
```

```
        self.breed = breed
        print(f"Con chó '{self.name}' giống {self.breed} đã được tạo.")

    def bark(self): # Phương thức riêng của Dog
        print(f"{self.name} sủa: Gâu gâu!")

class Cat(Animal): # Lớp con kế thừa từ Animal
    def __init__(self, name, color):
        super().__init__(name) # Gọi constructor của lớp cha
        self.color = color
        print(f"Con mèo '{self.name}' màu {self.color} đã được tạo.")

    def meow(self): # Phương thức riêng của Cat
        print(f"{self.name} kêu: Meo meo!")

# Tạo đối tượng
my_dog = Dog("Buddy", "Golden Retriever")
my_cat = Cat("Whiskers", "trắng")

# Các đối tượng con có thể sử dụng phương thức của lớp cha
my_dog.eat()    # Buddy đang ăn.
my_cat.sleep()  # Whiskers đang ngủ.

# Và các phương thức riêng của chúng
my_dog.bark()   # Buddy sủa: Gâu gâu!
my_cat.meow()   # Whiskers kêu: Meo meo!
```

Sử dụng super() để gọi constructor/phương thức của lớp cha

Khi một lớp con có phương thức __init__ riêng của nó, nó thường cần gọi phương thức __init__ của lớp cha để đảm bảo rằng các thuộc tính của lớp cha cũng được khởi tạo đúng cách. Python cung cấp hàm super() để làm điều này.

- super().__init__(*args, **kwargs): Gọi constructor của lớp cha.
- super().method_name(*args, **kwargs): Gọi một phương thức khác của lớp cha.

Tại sao dùng super()?

- **Đảm bảo khởi tạo đầy đủ:** Đảm bảo rằng tất cả các thuộc tính từ các lớp cha trong chuỗi kế thừa đều được khởi tạo.
- **Linh hoạt hơn trong đa kế thừa:** Khi có nhiều lớp cha, super() giúp quản lý thứ tự gọi phương thức (Method Resolution Order - MRO) một cách đúng đắn.

Ví dụ: (Đã dùng ở ví dụ trên với Animal/Dog/Cat)

9.2.3. Đa kế thừa (Multiple Inheritance - giới thiệu)

Đa kế thừa là khả năng một lớp con kế thừa từ nhiều lớp cha. Điều này cho phép lớp con kết hợp chức năng từ nhiều nguồn khác nhau.

Cú pháp:

```
class ChildClass(ParentClass1, ParentClass2, ...):
    pass
```

Ví dụ:

```
class Flyer:
    def fly(self):
        print("Tôi có thể bay.")

class Swimmer:
    def swim(self):
        print("Tôi có thể bơi.")

class Duck(Flyer, Swimmer): # Kế thừa từ cả Flyer và Swimmer
    def quack(self):
        print("Quack quack!")

my_duck = Duck()
my_duck.fly()
my_duck.swim()
my_duck.quack()
```

Lưu ý: Đa kế thừa có thể làm cho cấu trúc mã phức tạp và khó quản lý hơn do vấn đề "kim cương" (Diamond Problem) nếu các lớp cha có cùng tên phương thức. Trong Python, thứ tự ưu tiên được xác định bởi Method Resolution Order (MRO), có thể xem bằng ClassName.mro(). Thường thì các Design Pattern khác (như Composition) được ưu tiên hơn đa kế thừa để tránh phức tạp.

9.3. Tính chất của OOP

9.3.1. Tính đóng gói (Encapsulation)

Tính đóng gói (Encapsulation) là một trong những nguyên lý cơ bản của OOP, liên quan đến việc:

1. Gom dữ liệu và các phương thức hoạt động trên dữ liệu đó vào cùng một đơn vị (class).

2. Hạn chế quyền truy cập trực tiếp vào dữ liệu (thuộc tính) từ bên ngoài ˙, thay vào đó, việc truy cập và sửa đổi dữ liệu phải thông qua các phương thức công khai (public methods) được cung cấp bởi class đó.

Lợi ích của đóng gói:

- **Bảo vệ dữ liệu (Data Hiding):** Ngăn chặn việc thay đổi dữ liệu một cách không kiểm soát từ bên ngoài, giúp duy trì tính hợp lệ của trạng thái đối tượng.
- **Kiểm soát truy cập:** Cho phép nhà phát triển kiểm soát cách dữ liệu được truy cập và sửa đổi.
- **Dễ bảo trì và thay đổi:** Thay đổi cấu trúc nội bộ của class không ảnh hưởng đến các phần khác của chương trình miễn là giao diện công khai vẫn giữ nguyên.

Trong Python, không có khái niệm private hoặc protected thực sự như trong một số ngôn ngữ khác (ví dụ Java, C++). Tuy nhiên, Python có các quy ước để chỉ ra rằng một thuộc tính hoặc phương thức nên được coi là "nội bộ" và không nên truy cập trực tiếp từ bên ngoài:

1./ **Thuộc tính/Phương thức công khai (Public):** Không có tiền tố dấu gạch dưới. Có thể truy cập tự do từ bên ngoài Class.

```
class MyClass:
    def __init__(self, value):
        self.public_attribute = value # Công khai
    def public_method(self):
        print("Đây là một phương thức công khai.")
```

2./ **Thuộc tính/Phương thức "được bảo vệ" (Protected - theo quy ước):** Bắt đầu bằng một dấu gạch dưới (_). Quy ước này cho lập trình viên biết rằng đây là một thành phần nội bộ của Class và không nên truy cập trực tiếp từ bên ngoài, mặc dù Python vẫn cho phép bạn làm vậy.

```
class MyClass:
    def __init__(self, value):
        self._protected_attribute = value # Quy ước là protected
    def _protected_method(self):
        print("Đây là một phương thức được bảo vệ.")
```

Đây là một "dấu hiệu" cho các nhà phát triển khác biết rằng họ nên cẩn thận khi truy cập hoặc sửa đổi thành phần này.

3./ **Thuộc tính/Phương thức "riêng tư" (Private - theo quy ước/name mangling):** Bắt đầu bằng hai dấu gạch dưới (__). Python thực hiện "name mangling" cho các thuộc

tính/phương thức này, thay đổi tên của chúng để làm cho việc truy cập từ bên ngoài khó khăn hơn (nhưng không phải là không thể). Điều này chủ yếu để tránh xung đột tên trong các lớp con.

```
class MyClass:
    def __init__(self, value):
        self.__private_attribute = value # Quy ước là private (với name
mangling)
    def __private_method(self):
        print("Đây là một phương thức riêng tư.")

obj = MyClass(10)
# print(obj.__private_attribute) # Lỗi: AttributeError
# Cách truy cập qua name mangling (không nên làm)
# print(obj._MyClass__private_attribute) # Output: 10
```

Để truy cập hoặc sửa đổi các thuộc tính được "đóng gói", người ta thường sử dụng các phương thức **getter** (để lấy giá trị) và **setter** (để thiết lập giá trị).

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Quy ước là thuộc tính riêng tư

    def get_balance(self): # Getter
        return self.__balance

    def deposit(self, amount): # Phương thức công khai để thay đổi
balance
        if amount > 0:
            self.__balance += amount
            print(f"Đã nạp {amount}. Số dư mới: {self.__balance}")
        else:
            print("Số tiền nạp phải dương.")

    def withdraw(self, amount): # Phương thức công khai để thay đổi
balance
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Đã rút {amount}. Số dư mới: {self.__balance}")
        else:
            print("Số tiền rút không hợp lệ hoặc không đủ số dư.")
```

```
account = BankAccount(1000)
print(f"Số dư ban đầu: {account.get_balance()}")
account.deposit(500)
account.withdraw(200)
account.withdraw(2000) # Thử rút quá số dư
# print(account.__balance) # Lỗi: AttributeError
```

9.3.2. Tính đa hình (Polymorphism): ghi đè phương thức (Method Overriding)

Tính đa hình (Polymorphism) nghĩa là "nhiều hình thức". Trong OOP, nó đề cập đến khả năng các đối tượng thuộc các lớp khác nhau có thể phản ứng khác nhau với cùng một thông điệp (tức là cùng một phương thức). Điều này thường đạt được thông qua:

1. **Ghi đè phương thức (Method Overriding):** Lớp con cung cấp một triển khai cụ thể cho một phương thức đã được định nghĩa trong lớp cha. Khi phương thức đó được gọi trên một đối tượng của lớp con, phiên bản của lớp con sẽ được thực thi.
2. **Đa hình theo giao diện (Interface Polymorphism):** Các đối tượng khác nhau có thể được đối xử như cùng một loại nếu chúng chia sẻ một giao diện chung (ví dụ: cùng một tên phương thức), ngay cả khi chúng không có mối quan hệ kế thừa trực tiếp.

Ví dụ về Ghi đè phương thức (Method Overriding):

```
class Animal:
    def speak(self):
        print("Con vật kêu...") # Phương thức chung của lớp cha

class Dog(Animal):
    def speak(self): # Ghi đè phương thức speak() của lớp cha
        print("Gâu gâu!")

class Cat(Animal):
    def speak(self): # Ghi đè phương thức speak() của lớp cha
        print("Meo meo!")

class Duck(Animal):
    def speak(self): # Ghi đè phương thức speak() của lớp cha
        print("Quack quack!")
# Sử dụng đa hình
animals = [Dog(), Cat(), Duck(), Animal()] # Một list chứa các đối tượng
thuộc các lớp khác nhau
for animal in animals:
    animal.speak() # Mỗi đối tượng sẽ gọi phiên bản speak() riêng của nó
```

Output:

```
# Gâu gâu!
# Meo meo!
# Quack quack!
# Con vật kêu...
```

Ở đây, cùng một lời gọi animal.speak() nhưng kết quả khác nhau tùy thuộc vào loại đối tượng mà animal đang tham chiếu đến.

9.3.3. Các nguyên lý SOLID (giới thiệu ý tưởng)

SOLID là một tập hợp 5 nguyên lý thiết kế hướng đối tượng được giới thiệu bởi Robert C. Martin. Tuân thủ các nguyên lý này giúp viết mã dễ bảo trì, dễ mở rộng và dễ hiểu hơn.

1. S - Single Responsibility Principle (SRP - Nguyên lý Trách nhiệm đơn nhất):
   - Mỗi class chỉ nên có một và chỉ một lý do để thay đổi.
   - Nói cách khác, một class chỉ nên có một trách nhiệm duy nhất.
   - Ví dụ:
     - Một class User chỉ nên quản lý thông tin User, không nên xử lý việc lưu trữ User vào database hay gửi email cho User.
2. O - Open/Closed Principle (OCP - Nguyên lý Mở/Đóng):
   - Các thực thể phần mềm (class, module, hàm, v.v.) nên **mở để mở rộng, nhưng đóng để sửa đổi**.
   - Nghĩa là bạn nên có thể thêm chức năng mới mà không cần thay đổi mã nguồn hiện có của các class đã hoạt động tốt. Kế thừa và đa hình là công cụ chính cho nguyên lý này.
3. L - Liskov Substitution Principle (LSP - Nguyên lý Thay thế Liskov):
   - Các đối tượng của class con phải có thể **thay thế cho các đối tượng của class cha** mà không làm hỏng tính đúng đắn của chương trình.
   - Tức là, nếu S là một class con của T, thì các đối tượng của T có thể được thay thế bằng các đối tượng của S mà không làm thay đổi các tính chất mong muốn của chương trình.
4. I - Interface Segregation Principle (ISP - Nguyên lý Tách biệt Giao diện):
   - Các client (người dùng) không nên bị buộc phải phụ thuộc vào các phương thức mà họ không sử dụng.

- Thay vì một giao diện lớn, "phì nhiêu", nên có nhiều giao diện nhỏ, chuyên biệt hơn.

5. D - Dependency Inversion Principle (DIP - Nguyên lý Đảo ngược Phụ thuộc):
   - Các module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào các trừu tượng (abstractions).
   - Trừu tượng (interfaces/abstract classes) không nên phụ thuộc vào các chi tiết. Các chi tiết (implementations) nên phụ thuộc vào các trừu tượng.
   - Mục tiêu là giảm sự phụ thuộc cứng nhắc giữa các thành phần.

Các nguyên lý SOLID là kim chỉ nam giúp bạn thiết kế các hệ thống hướng đối tượng mạnh mẽ, dễ bảo trì và có khả năng mở rộng tốt trong tương lai.

BÀI TẬP:

1. Thiết kế một **Class Book**. Class này có các **thuộc tính** (title, author, price) và một **phương thức** display_info() in ra thông tin chi tiết. Tạo 2 đối tượng Book và gọi phương thức display_info().
2. Thiết kế Class **BankAccount** với thuộc tính __balance (private) và phương thức deposit(), withdraw().
3. Thêm phương thức **đặc biệt __str__** (dunder method) vào Class BankAccount để khi in đối tượng (print(account)), nó hiển thị trạng thái số dư một cách thân thiện.
4. Xây dựng Class **Library**. Thuộc tính là một **List** các đối tượng Book. Phương thức add_book() (thêm sách) và list_books() (in ra tất cả sách).

BÀI TẬP:

1. Kế thừa từ Class Book (Tuần 8) để tạo Class Ebook. Thêm thuộc tính file_size. Ghi đè phương thức display_info() để hiển thị cả kích thước file.
2. Thêm thuộc tính __discount_rate (private) cho Class Book. Viết phương thức get_discounted_price để truy cập và tính giá đã giảm.

THỰC HÀNH 09: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

9.1. Xây dựng Class & Object đơn giản

9.1.1. Thiết kế và cài đặt các lớp đơn giản:

Student (tên, tuổi, điểm), Car (hãng, mẫu, tốc độ)

Mục tiêu: Áp dụng kiến thức về class, Object, thuộc tính và phương thức.

Bài tập: Tạo file basic_oop.py.

1. Class Student:

– **Thuộc tính:** name (chuỗi), age (số nguyên), grade (số thực).

– Phương thức:

- __init__(self, name, age, grade): Khởi tạo các thuộc tính.
- display_info(self): In ra thông tin chi tiết của học sinh.
- is_passing(self, passing_grade=5.0): Trả về True nếu grade lớn hơn hoặc bằng passing_grade, ngược lại là False.

2. Class Car:

– **Thuộc tính:** make (hãng - chuỗi), model (mẫu - chuỗi), speed (tốc độ hiện tại - số nguyên, mặc định 0).

– Phương thức:

- __init__(self, make, model): Khởi tạo make, model, và speed mặc định là 0.
- accelerate(self, amount): Tăng speed thêm amount. Không cho phép speed vượt quá 200.
- brake(self, amount): Giảm speed đi amount. Không cho phép speed xuống dưới 0.
- get_info(self): Trả về một chuỗi chứa thông tin hãng, mẫu và tốc độ hiện tại.

9.1.2. Tạo các đối tượng từ các lớp này và truy cập thuộc tính, gọi phương thức

Mục tiêu: Thực hành tạo và tương tác với đối tượng.

Bài tập tiếp theo trong basic_oop.py:

1. Tạo hai đối tượng Student: student1 và student2 với các thông tin khác nhau.
2. Gọi phương thức display_info() và is_passing() cho từng đối tượng Student.
3. Tạo hai đối tượng Car: car1 và car2.

4. Gọi phương thức accelerate() và brake() cho car1 nhiều lần, kiểm tra giá trị tốc độ.
5. In thông tin của car1 và car2 bằng phương thức get_info().

Gợi ý:

```
# basic_oop.py

print("--- THỰC HÀNH XÂY DỰNG CLASS & OBJECT ĐƠN GIẢN ---")

# 1. Class Student
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade # ví dụ: 0.0 - 10.0

    def display_info(self):
        print(f"Học sinh: {self.name}, Tuổi: {self.age}, Điểm:
{self.grade:.1f}")

    def is_passing(self, passing_grade=5.0):
        return self.grade >= passing_grade

# 2. Class Car
class Car:
    def __init__(self, make, model):
        self.make = make
        self.model = model
        self.speed = 0 # Tốc độ mặc định khi khởi tạo xe

    def accelerate(self, amount):
        if amount > 0:
            self.speed += amount
            if self.speed > 200:
                self.speed = 200 # Giới hạn tốc độ tối đa
            print(f"{self.make} {self.model} tăng tốc. Tốc độ hiện tại:
{self.speed} km/h")
        else:
            print("Lượng tăng tốc phải dương.")

    def brake(self, amount):
```

```
        if amount > 0:
            self.speed -= amount
            if self.speed < 0:
                self.speed = 0 # Tốc độ không thể âm
            print(f"{self.make} {self.model} giảm tốc. Tốc độ hiện tại:
{self.speed} km/h")
        else:
            print("Lượng giảm tốc phải dương.")

    def get_info(self):
        return f"Xe: {self.make} {self.model}, Tốc độ: {self.speed} km/h"

# --- Tạo đối tượng và tương tác ---

print("\n--- THỰC HÀNH VỚI STUDENT ---")
student1 = Student("Tran Van C", 18, 7.5)
student2 = Student("Le Thi D", 19, 4.8)

student1.display_info()
print(f"Học sinh {student1.name} có đậu không? {student1.is_passing()}")
# Mặc định passing_grade=5.0

student2.display_info()
print(f"Học sinh {student2.name} có đậu không?
{student2.is_passing(6.0)}") # Với passing_grade=6.0
print("\n--- THỰC HÀNH VỚI CAR ---")
car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "CR-V")

print(car1.get_info())
car1.accelerate(50)
car1.accelerate(80)
car1.accelerate(100) # Đáng lẽ sẽ đạt 230, nhưng bị giới hạn 200
car1.brake(60)
print(car1.get_info())

print(car2.get_info())
car2.accelerate(70)
car2.brake(100) # Đáng lẽ sẽ xuống -30, nhưng bị giới hạn 0
print(car2.get_info())
```

9.2. Ứng dụng Kế thừa

9.2.1. Xây dựng hệ thống lớp Animal (lớp cha) và các lớp con như Dog, Cat (kế thừa các thuộc tính và phương thức chung, thêm các thuộc tính riêng).

Mục tiêu: Thực hành kế thừa đơn giản.

Bài tập: Tạo file inheritance_animal.py.

1. Lớp cha Animal:
   - **Thuộc tính:** name (chuỗi).
   - Phương thức:
     - __init__(self, name): Khởi tạo name.
     - make_sound(self): In ra "Some generic sound".
2. Lớp con Dog (kế thừa từ Animal):
   - Thuộc tính riêng: breed (chuỗi).
   - Phương thức:
     - __init__(self, name, breed): Gọi constructor của lớp cha và khởi tạo breed.
     - make_sound(self): Ghi đè phương thức của lớp cha, in ra "Woof! Woof!".
     - fetch(self): In ra "{name} is fetching the ball."
3. Lớp con Cat (kế thừa từ Animal):
   - **Thuộc tính riêng:** fur_color (chuỗi).
   - Phương thức:
     - __init__(self, name, fur_color): Gọi constructor của lớp cha và khởi tạo fur_color.
     - make_sound(self): Ghi đè phương thức của lớp cha, in ra "Meow!".
     - scratch_furniture(self): In ra "{name} is scratching the furniture."

9.3. Thực hành gọi phương thức của lớp cha từ lớp con

Mục tiêu: Áp dụng super().

Bài tập tiếp theo trong inheritance_animal.py:

1. Tạo một đối tượng Dog và một đối tượng Cat.
2. Gọi phương thức make_sound() cho cả hai đối tượng.

3. Gọi các phương thức riêng của từng lớp con (fetch() cho Dog, scratch_furniture() cho Cat).
4. (Tùy chọn) Trong phương thức make_sound của Dog hoặc Cat, thử gọi super().make_sound() trước khi in ra âm thanh riêng của chúng để thấy cách lớp cha được gọi.

9.3.1. Thiết kế lớp Shape và các lớp con Circle, Rectangle với phương thức tính diện tích

Mục tiêu: Áp dụng kế thừa và đa hình cho bài toán hình học.

Bài tập tiếp theo trong inheritance_animal.py (hoặc một file mới inheritance_shape.py):

1. **Lớp cha Shape:**
   - **Phương thức:** __init__(self, color)
   - **Phương thức:** get_color(self)
   - **Phương thức:** calculate_area(self): Trả về 0 hoặc báo lỗi NotImplementedError (vì lớp cha không biết cách tính diện tích cụ thể).
2. **Lớp con Circle (kế thừa từ Shape):**
   - **Thuộc tính riêng:** radius (số thực).
   - Phương thức:
     - __init__(self, color, radius): Gọi constructor của lớp cha và khởi tạo radius.
     - calculate_area(self): Ghi đè phương thức, tính diện tích hình tròn (π×radius2).
3. **Lớp con Rectangle (kế thừa từ Shape):**
   - **Thuộc tính riêng:** width (số thực), height (số thực).
   - Phương thức:
     - __init__(self, color, width, height): Gọi constructor của lớp cha và khởi tạo width, height.
     - calculate_area(self): Ghi đè phương thức, tính diện tích hình chữ nhật (width×height).
4. **Thực hiện:**
     - Tạo các đối tượng Circle và Rectangle.
     - Gọi phương thức calculate_area() cho từng đối tượng và in ra diện tích.
     - Tạo một List chứa các đối tượng Shape khác nhau (Circle, Rectangle) và lặp qua chúng để gọi calculate_area() (minh họa đa hình).

Gợi ý:

```
# inheritance_animal.py hoặc chia thành 2 file

print("--- THỰC HÀNH KẾ THỪA: ANIMAL ---")

# Lớp cha Animal
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal '{self.name}' created.")

    def make_sound(self):
        print("Some generic sound.")

# Lớp con Dog
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # Gọi constructor của lớp cha Animal
        self.breed = breed
        print(f"Dog '{self.name}' ({self.breed}) created.")

    def make_sound(self): # Ghi đè phương thức
        # super().make_sound() # Có thể gọi phương thức của lớp cha nếu
muốn
        print("Woof! Woof!")

    def fetch(self):
        print(f"{self.name} is fetching the ball.")

# Lớp con Cat
class Cat(Animal):
    def __init__(self, name, fur_color):
        super().__init__(name) # Gọi constructor của lớp cha Animal
        self.fur_color = fur_color
        print(f"Cat '{self.name}' ({self.fur_color} fur) created.")

    def make_sound(self): # Ghi đè phương thức
        print("Meow! Meow!")

    def scratch_furniture(self):
```

```
        print(f"{self.name} is scratching the furniture.")

# --- Thực hành tạo và gọi phương thức ---
print("\n--- TEST ANIMAL CLASSES ---")
my_dog = Dog("Rex", "German Shepherd")
my_cat = Cat("Luna", "White")

my_dog.make_sound()
my_cat.make_sound()

my_dog.fetch()
my_cat.scratch_furniture()

print("\n--- THỰC HÀNH KẾ THỪA & ĐA HÌNH: SHAPE ---")
import math

# Lớp cha Shape
class Shape:
    def __init__(self, color):
        self.color = color

    def get_color(self):
        return self.color

    def calculate_area(self):
        # Đây là một phương thức trừu tượng, lớp con phải ghi đè nó
        # Nếu không ghi đè và gọi nó, sẽ báo lỗi
        raise NotImplementedError("Subclasses must implement
calculate_area method.")

# Lớp con Circle
class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color)
        if radius <= 0:
            raise ValueError("Bán kính phải dương.")
        self.radius = radius

    def calculate_area(self): # Ghi đè phương thức calculate_area
        return math.pi * (self.radius ** 2)
```

```
# Lớp con Rectangle
class Rectangle(Shape):
    def __init__(self, color, width, height):
        super().__init__(color)
        if width <= 0 or height <= 0:
            raise ValueError("Chiều rộng và chiều cao phải dương.")
        self.width = width
        self.height = height

    def calculate_area(self): # Ghi đè phương thức calculate_area
        return self.width * self.height

# --- Thực hành tạo đối tượng và tính diện tích ---
print("\n--- TEST SHAPE CLASSES ---")
circle1 = Circle("Red", 5)
rectangle1 = Rectangle("Blue", 4, 6)

print(f"Hình tròn màu {circle1.get_color()} với bán kính
{circle1.radius}. Diện tích: {circle1.calculate_area():.2f}")
print(f"Hình chữ nhật màu {rectangle1.get_color()} với chiều rộng
{rectangle1.width}, chiều cao {rectangle1.height}. Diện tích:
{rectangle1.calculate_area()}")

# Minh họa tính đa hình
shapes = [Circle("Green", 3), Rectangle("Yellow", 2, 8), Circle("Purple",
1.5)]
print("\n--- Tính diện tích các hình từ một list ---")
for shape in shapes:
    # Python sẽ tự động gọi phương thức calculate_area() phù hợp với từng
loại đối tượng
    print(f"Diện tích của hình {shape.get_color()}:
{shape.calculate_area():.2f}")

# try:
#     generic_shape = Shape("Black")
#     generic_shape.calculate_area() # Sẽ gây lỗi NotImplementedError
# except NotImplementedError as e:
#     print(f"\nCaught an error: {e}")
```

9.4. Tổng hợp OOP

9.4.1. Viết một chương trình nhỏ mô phỏng hệ thống quản lý thư viện hoặc tài khoản ngân hàng sử dụng các khái niệm OOP đã học

Mục tiêu: Kết hợp Class, Object, Constructor, Method và Kế thừa để xây dựng một ứng dụng nhỏ.

Bài tập: Tạo file library_or_bank_system.py.

Chọn một trong hai kịch bản sau:

Kịch bản 1: Hệ thống quản lý thư viện đơn giản

1. Class Book:
   - **Thuộc tính:** title, author, isbn, is_borrowed (boolean, mặc định False).
   - Phương thức:
     - __init__(self, title, author, isbn): Khởi tạo các thuộc tính.
     - display_info(self): In thông tin sách và trạng thái mượn.
     - borrow(self): Đặt is_borrowed thành True nếu sách chưa được mượn, thông báo.
     - return_book(self): Đặt is_borrowed thành False nếu sách đang được mượn, thông báo.
2. Class Library:
   - **Thuộc tính:** books (một dictionary, khóa là isbn, giá trị là đối tượng Book).
   - Phương thức:
     - __init__(self): Khởi tạo books rỗng.
     - add_book(self, book_obj): Thêm đối tượng Book vào thư viện (kiểm tra isbn trùng lặp).
     - find_book(self, isbn): Tìm sách theo isbn, trả về đối tượng Book hoặc None.
     - list_all_books(self): In ra thông tin của tất cả sách trong thư viện.
     - borrow_book(self, isbn): Gọi phương thức borrow() của đối tượng sách.
     - return_book(self, isbn): Gọi phương thức return_book() của đối tượng sách.
3. Tạo chương trình chính:
   - Tạo một đối tượng Library.

- Thêm một vài cuốn sách mẫu.
- Thực hiện các thao tác mượn/trả, tìm kiếm, liệt kê.

Kịch bản 2: Hệ thống tài khoản ngân hàng đơn giản

1. Class Account:
   - **Thuộc tính:** account_number (chuỗi), account_holder (chuỗi), _balance (số thực, sử dụng _ để gợi ý bảo vệ).
   - Phương thức:
     - __init__(self, account_number, account_holder, initial_balance=0.0): Khởi tạo các thuộc tính.
     - get_balance(self): Trả về số dư hiện tại.
     - deposit(self, amount): Nạp tiền (kiểm tra số tiền dương), cập nhật _balance.
     - withdraw(self, amount): Rút tiền (kiểm tra số tiền dương và đủ số dư), cập nhật _balance.
     - display_info(self): In thông tin tài khoản và số dư.
2. Class SavingsAccount (kế thừa từ Account):
   - **Thuộc tính riêng:** interest_rate (số thực, ví dụ 0.05 cho 5%).
   - Phương thức:
     - __init__(self, account_number, account_holder, initial_balance=0.0, interest_rate=0.01): Gọi constructor lớp cha và khởi tạo interest_rate.
     - add_interest(self): Cộng lãi vào số dư (_balance += _balance * interest_rate).
3. Class CheckingAccount (kế thừa từ Account):
   - **Thuộc tính riêng: overdraft**_limit (số thực, ví dụ 1000).
   - Phương thức:
     - __init__(self, account_number, account_holder, initial_balance=0.0, overdraft_limit=0.0): Gọi constructor lớp cha và khởi tạo overdraft_limit.
     - withdraw(self, amount): Ghi đè phương thức withdraw. Cho phép rút quá số dư đến giới hạn overdraft_limit.
4. Tạo chương trình chính:
   - Tạo một đối tượng SavingsAccount và một CheckingAccount.

- Thực **hiện** các thao tác nạp, rút, kiểm tra số dư, cộng lãi (cho tiết kiệm), và rút quá hạn mức (cho tài khoản vãng lai).

9.4.2. Thực hành đóng gói dữ liệu (nếu có thể)

Mục tiêu: Áp dụng quy ước đóng gói bằng dấu gạch dưới (_ hoặc __) và sử dụng getter/setter.

Gợi ý: (Đã tích hợp trong các kịch bản trên, ví dụ _balance và các phương thức deposit, withdraw, get_balance).

Gợi ý cho Kịch bản 2 (Hệ thống tài khoản ngân hàng):

```
# library_or_bank_system.py (Chọn kịch bản Bank System)

print("--- HỆ THỐNG QUẢN LÝ TÀI KHOẢN NGÂN HÀNG ---")

# Lớp cha Account
class Account:
    def __init__(self, account_number, account_holder,
initial_balance=0.0):
        self.account_number = account_number
        self.account_holder = account_holder
        self._balance = initial_balance # Sử dụng _ để gợi ý đây là thuộc
tính "được bảo vệ"

    def get_balance(self):
        """Trả về số dư hiện tại."""
        return self._balance

    def deposit(self, amount):
        """Nạp tiền vào tài khoản."""
        if amount > 0:
            self._balance += amount
            print(f"Tài khoản {self.account_number}: Đã nạp {amount:,.2f}
VNĐ. Số dư mới: {self._balance:,.2f} VNĐ")
        else:
            print("Số tiền nạp phải là số dương.")

    def withdraw(self, amount):
        """Rút tiền từ tài khoản."""
        if amount <= 0:
            print("Số tiền rút phải là số dương.")
```

```
            return False
        if amount > self._balance:
            print("Số dư không đủ để rút tiền.")
            return False

        self._balance -= amount
        print(f"Tài khoản {self.account_number}: Đã rút {amount:,.2f}
VNĐ. Số dư mới: {self._balance:,.2f} VNĐ")
        return True

    def display_info(self):
        """In thông tin tài khoản và số dư."""
        print(f"\n--- Thông tin Tài khoản ---")
        print(f"Số tài khoản: {self.account_number}")
        print(f"Chủ tài khoản: {self.account_holder}")
        print(f"Số dư: {self._balance:,.2f} VNĐ")
        print(f"----------------------------")

# Lớp con SavingsAccount (Kế thừa từ Account)
class SavingsAccount(Account):
    def __init__(self, account_number, account_holder,
initial_balance=0.0, interest_rate=0.01):
        super().__init__(account_number, account_holder, initial_balance)
        if not (0 <= interest_rate <= 1):
             raise ValueError("Lãi suất phải nằm trong khoảng 0 đến 1.")
        self.interest_rate = interest_rate
        print(f"Tài khoản tiết kiệm {self.account_number} đã được tạo.")

    def add_interest(self):
        """Cộng lãi vào số dư."""
        interest_amount = self._balance * self.interest_rate
        self._balance += interest_amount
        print(f"Tài khoản {self.account_number}: Đã cộng lãi
{interest_amount:,.2f} VNĐ. Số dư mới: {self._balance:,.2f} VNĐ")

    def display_info(self): # Ghi đè để thêm thông tin lãi suất
        super().display_info()
        print(f"Loại tài khoản: Tiết kiệm")
        print(f"Lãi suất: {self.interest_rate * 100:.2f}%")

# Lớp con CheckingAccount (Kế thừa từ Account)
```

```
class CheckingAccount(Account):
    def __init__(self, account_number, account_holder,
initial_balance=0.0, overdraft_limit=0.0):
        super().__init__(account_number, account_holder, initial_balance)
        if overdraft_limit < 0:
            raise ValueError("Hạn mức thấu chi không thể âm.")
        self.overdraft_limit = overdraft_limit
        print(f"Tài khoản vãng lai {self.account_number} đã được tạo.")

    def withdraw(self, amount): # Ghi đè phương thức withdraw
        """Rút tiền từ tài khoản, cho phép thấu chi."""
        if amount <= 0:
            print("Số tiền rút phải là số dương.")
            return False

        # Kiểm tra nếu số tiền rút vượt quá số dư hiện có nhưng vẫn trong
hạn mức thấu chi
        if self._balance - amount >= -self.overdraft_limit:
            self._balance -= amount
            print(f"Tài khoản {self.account_number}: Đã rút {amount:,.2f}
VNĐ. Số dư mới: {self._balance:,.2f} VNĐ")
            return True
        else:
            print(f"Số tiền rút vượt quá hạn mức thấu chi cho phép
({self.overdraft_limit:,.2f} VNĐ).")
            return False

    def display_info(self): # Ghi đè để thêm thông tin hạn mức thấu chi
        super().display_info()
        print(f"Loại tài khoản: Vãng lai")
        print(f"Hạn mức thấu chi: {self.overdraft_limit:,.2f} VNĐ")


# --- Chương trình chính ---
print("\n--- TEST HỆ THỐNG NGÂN HÀNG ---")

# Tạo tài khoản tiết kiệm
savings1 = SavingsAccount("S001", "Nguyen Van A", 1000000, 0.03) # 1
triệu đồng, 3% lãi suất
savings1.display_info()
savings1.deposit(200000)
```

```
savings1.withdraw(50000)
savings1.add_interest()
savings1.display_info()

# Tạo tài khoản vãng lai
checking1 = CheckingAccount("C001", "Tran Thi B", 500000, 100000) # 500k,
thấu chi 100k
checking1.display_info()
checking1.withdraw(300000) # Rút trong giới hạn số dư
checking1.withdraw(300000) # Rút quá số dư, nhưng trong hạn mức thấu chi
(-100k)
checking1.display_info()
checking1.withdraw(50000) # Rút quá hạn mức thấu chi
checking1.deposit(100000)
checking1.display_info()
```

BÀI TẬP:

1. Kế thừa từ BankAccount (Tuần 8) để tạo Class **SavingAccount**. Ghi đè phương thức deposit() để luôn thêm một khoản **tiền thưởng** nhỏ (ví dụ: 1%) vào số dư.
2. Áp dụng **đa hình (Polymorphism)**: Thiết kế một hàm **process_transaction(account)** có thể nhận cả đối tượng BankAccount và SavingAccount làm đối số, và gọi phương thức withdraw() của chúng.
3. Thiết kế 3 lớp lồng nhau (Parent, Child1, Child2) minh họa **kế thừa đa cấp**. Sử dụng hàm super().__init__() để đảm bảo khởi tạo đúng lớp cha.